Network_Traffic_Analysis_Report

1. Objective

The objective of this challenge was to analyze the supplied PCAP file, identify the communication between the client and server, recover any sensitive information transmitted over the network, identify weaknesses, and retrieve the hidden CTF flag.

2. Scope

The assessment was limited to the provided PCAP file. No active interaction with the target system was performed.

3. Tools Used

- Wireshark
- CyberChef
- Python3
- Kali Linux

4. Methodology

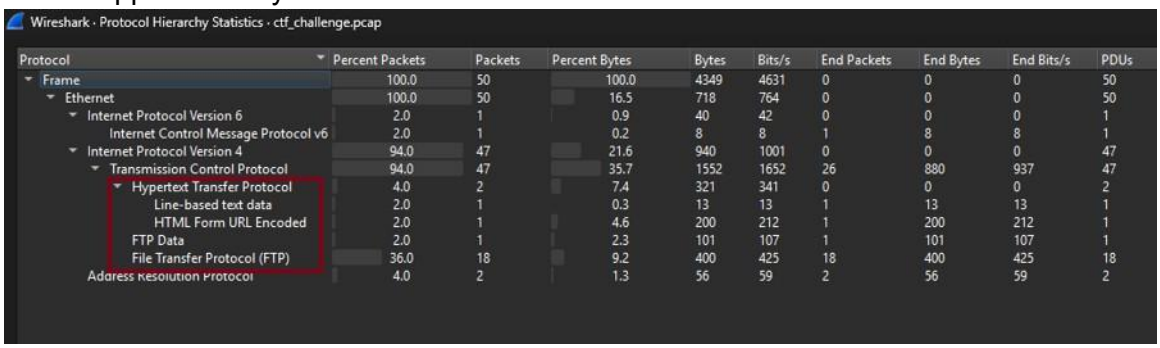
4.1 Initial Traffic Inspection

Steps Performed

1. Opened the provided PCAP file in Wireshark.
2. Reviewed the Protocol Hierarchy to identify the protocols present.
3. Observed ARP, FTP, and HTTP traffic.
4. Used Follow TCP Stream to reconstruct the network conversations.

Findings

- The capture contained IPv4, ARP, FTP, and HTTP traffic.
- FTP and HTTP were selected for detailed analysis because they contained application-layer communication.



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	50	100.0	4349	4631	0	0	0	50
Ethernet	100.0	50	16.5	718	764	0	0	0	50
Internet Protocol Version 6	2.0	1	0.9	40	42	0	0	0	1
Internet Control Message Protocol v6	2.0	1	0.2	8	8	1	8	8	1
Internet Protocol Version 4	94.0	47	21.6	940	1001	0	0	0	47
Transmission Control Protocol	94.0	47	35.7	1552	1652	26	880	937	47
Hypertext Transfer Protocol	4.0	2	7.4	321	341	0	0	0	2
Line-based text data	2.0	1	0.3	13	13	1	13	13	1
HTML Form URL Encoded	2.0	1	4.6	200	212	1	200	212	1
FTP Data	2.0	1	2.3	101	107	1	101	107	1
File Transfer Protocol (FTP)	36.0	18	9.2	400	425	18	400	425	18
Address Resolution Protocol	4.0	2	1.3	56	59	2	56	59	2

Figure 1. Protocol hierarchy of the captured network traffic.

4.2 ARP Analysis

Steps Performed

1. Examined the initial ARP packets in the packet capture.
2. Identified the source and destination IP addresses.
3. Reviewed the ARP request and corresponding ARP reply.
4. Verified that the client successfully resolved the MAC address before initiating TCP communication.

Findings

The packet capture begins with an ARP exchange. The client (**10.0.2.15**) sent the following ARP request:

- **Who has 10.0.2.1? Tell 10.0.2.15**

The device with IP address **10.0.2.1** replied:

- **10.0.2.1 is at 52:54:00:12:35:00**

This ARP exchange successfully resolved the MAC address required for subsequent network communication. No suspicious activity or sensitive information was observed during the ARP exchange.

4.3 TCP Connection Establishment Steps Performed

1. Applied the Wireshark display filter **tcp.stream == 0** to isolate the FTP control connection.
2. Examined the initial TCP packets exchanged between the client and the server.
3. Verified the TCP three-way handshake before the FTP session was established.

Findings

The client 10.0.2.15 initiated a TCP connection from the ephemeral source port 60112 to the FTP server 10.0.2.4 on destination port 21, the default FTP control port.

The connection was successfully established using the standard TCP three-way handshake:

- SYN – Client → Server
- SYN, ACK – Server → Client
- ACK – Client → Server

Role	IP Address	Port
Client	10.0.2.15	60112
FTP Server	10.0.2.4	21

After the TCP handshake was completed, the FTP server responded with its banner, indicating that the FTP service was ready to accept client requests.

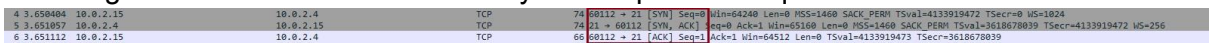


Figure 2. TCP three-way handshake establishing the FTP control connection.

4.4 FTP Analysis

Steps Performed

1. Applied the Wireshark display filter: **tcp.stream == 0**
2. Followed the FTP control session using **Follow → TCP Stream**.
3. Examined the FTP banner returned by the server.
4. Reviewed the authentication process and FTP commands.

Findings

1. FTP Banner and Authentication

After the TCP connection was established, the FTP server returned the following banner:

220 pyftpdlib 2.2.0 ready.

This indicates that the FTP service was running **pyftpdlib 2.2.0**, a Python-based FTP server library.

The client then authenticated using:

USER anonymous
PASS -wget@
230 Login successful.

indicating that anonymous authentication was permitted.

Figure 3. FTP control session showing the server banner, anonymous login, and successful authentication.

2. FTP Commands and Passive Mode

Following authentication, the client executed several FTP commands:

SYST
PWD
TYPE I
SIZE agent_config.json

The client then entered Passive Mode using the **PASV** command:

227 Entering passive mode (10,0,2,4,177,141)

The values (177,141) correspond to TCP port **45453**, which was used as the FTP data channel.

3. Passive Data Connection

After the PASV command, the client established a second TCP connection to port 45453. Unlike the control connection on TCP port 21, this connection was dedicated solely to transferring the file contents. The client requested the file using:

RETR agent_config.json The server responded:

125 Data connection already open. Transfer starting. 226 Transfer complete.

confirming that the file was successfully transferred.

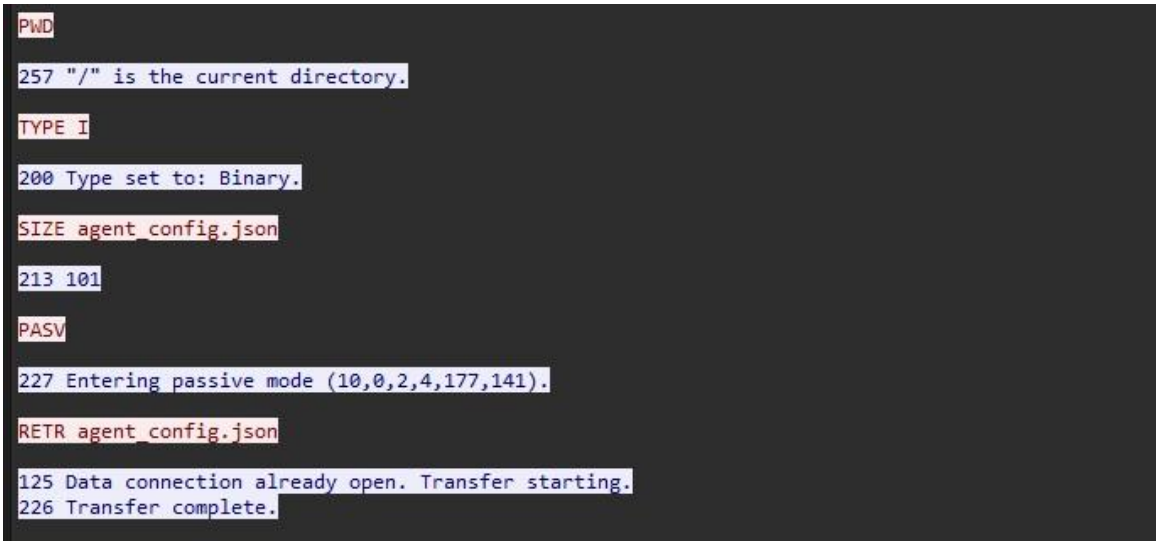


Figure 4. FTP commands showing directory enumeration, binary transfer mode, file size request, passive mode negotiation, and retrieval of agent_config.json.

4.5 Configuration File Analysis

Steps Performed

1. Used **File → Export Objects → FTP-DATA** to identify files transferred over the FTP data connection.
2. Exported the recovered file **agent_config.json**.
3. Opened the file in a text editor to examine its contents.
4. Verified the same contents using **Follow → TCP Stream** on the FTP-DATA connection.

Findings

The FTP data transfer contained a configuration file named **agent_config.json**. The contents of the recovered file were:

```
{
   "status": "staged",
   "aes_key": "SuperSecretKey1234567890123456",
   "iv": "InitVector123456"
}
```

The configuration file contained three important values:

- status – Indicates that the client was in the staged state, suggesting it was waiting for further instructions from the server.
- aes_key – A 32-character string intended to be used as the AES encryption key.
- iv – The initialization vector used during AES encryption.

These values became the primary evidence for analyzing the encrypted HTTP payload observed later in the packet capture.

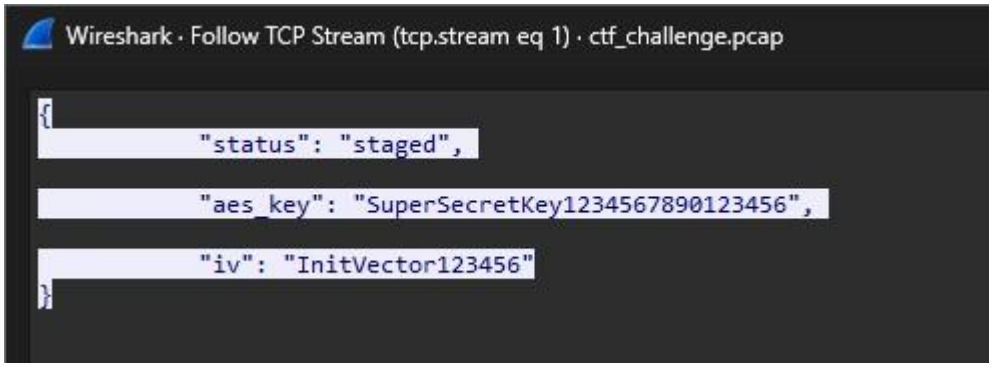


Figure 5. Contents of the recovered agent_config.json file extracted from the FTP data stream.

4.6 HTTP Data Transmission Analysis

Steps Performed

1. Applied the Wireshark display filter **http.**
2. Followed the HTTP conversation using:
3. Follow → HTTP Stream
4. Examined the HTTP request sent by the client.
5. Inspected the transmitted POST data.
6. Reviewed the HTTP response returned by the server.

Findings

After downloading **agent_config.json** via FTP, the client communicated with the web server over HTTP.

The client sent an **HTTP POST** request to the **/data** endpoint with the following details:

- **Method:** POST

- **URI:** /data
- **Content-Type:** application/x-www-form-urlencoded
- **User-Agent:** curl/8.19.0

The request body contained a parameter named **payload**, which held a long Base64encoded string, indicating that the transmitted data was encrypted or encoded.
The server responded with **HTTP/1.1 200 OK** and the message **"Data received"**, confirming that the request was successfully processed.
Since the previously recovered **agent_config.json** contained an AES key and IV, the captured payload became the primary target for further decryption and analysis.

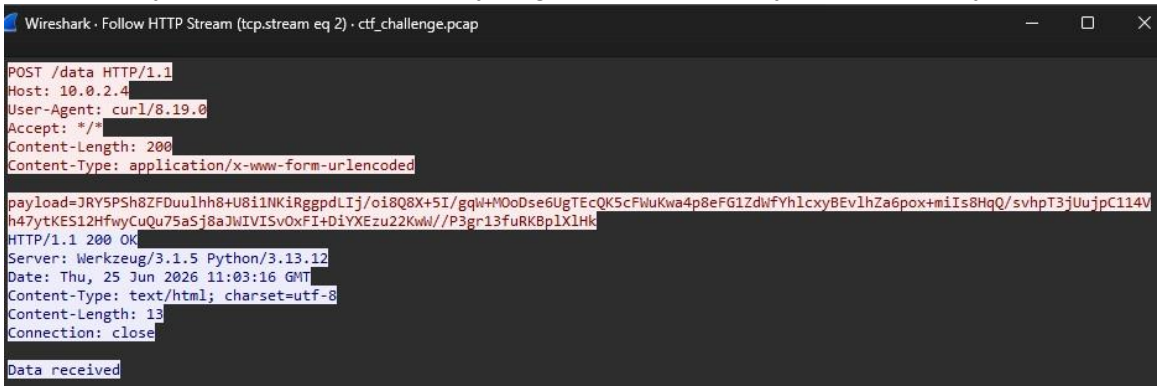


Figure 6. HTTP POST request containing the encoded/encrypted payload and the server's "Data received" response observed using Follow HTTP Stream.

5. Vulnerability Identification

5.1 Anonymous FTP Access

Allowing anonymous FTP access enables any user to connect without valid credentials and download files exposed by the server. In this challenge, the configuration file agent_config.json was successfully retrieved through anonymous authentication.
Severity: Medium

5.2 Sensitive Information Stored in Plaintext

The AES encryption key and initialization vector were stored in plaintext and transferred directly to the client. Anyone intercepting the network traffic could recover these values.
Severity: High

5.3 Unencrypted FTP Communication

FTP does not encrypt authentication or transferred files. Any attacker monitoring the network could capture credentials (if used) and transferred files.
Severity: High

5.4 HTTP Communication Without TLS

Although the payload appeared encrypted, the HTTP protocol exposes request headers, URLs, metadata, and traffic patterns to anyone monitoring the network.
Severity: Medium

5.5 Exposure of Cryptographic Material

Even if the payload is encrypted, exposing the encryption key allows an attacker to attempt offline decryption of intercepted communications.
Severity: High

6. Exploitation Process

6.1 Recovering the Encryption Parameters

The FTP analysis revealed that the file **agent_config.json** contained the following configuration:

```
{
   "status": "staged",
   "aes_key": "SuperSecretKey1234567890123456",
   "iv": "InitVector123456"
}
```

The HTTP POST request captured in the PCAP contained a parameter named **payload**, whose value was a Base64-encoded string.

Based on the presence of an AES key and IV in the configuration file, I hypothesized that the HTTP payload had been encrypted using AES before being transmitted.

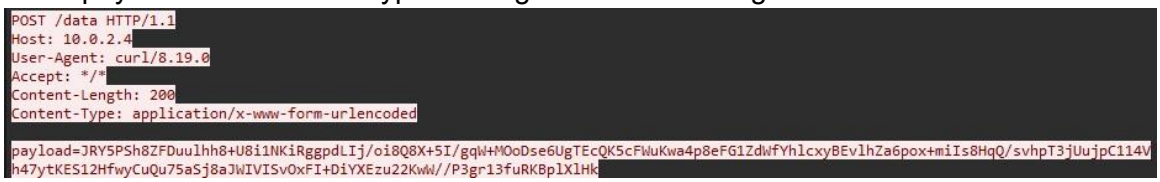


Figure 7. HTTP POST request containing the encrypted payload.

6.2 AES Decryption

To verify the hypothesis, I used **CyberChef**.

Steps Performed

- Copied the Base64-encoded payload from the HTTP POST request.
- Added the **From Base64** operation.
- Added the **AES Decrypt** operation.
- Configured:
  - Mode: **CBC**
  - IV: InitVector123456 ○ Input: Raw ○ Output: Raw

Initially, CyberChef reported:

Invalid key length: 30 bytes

AES requires a key length of **16**, **24**, or **32** bytes. The recovered key was only **30 bytes**, so it could not be used directly.

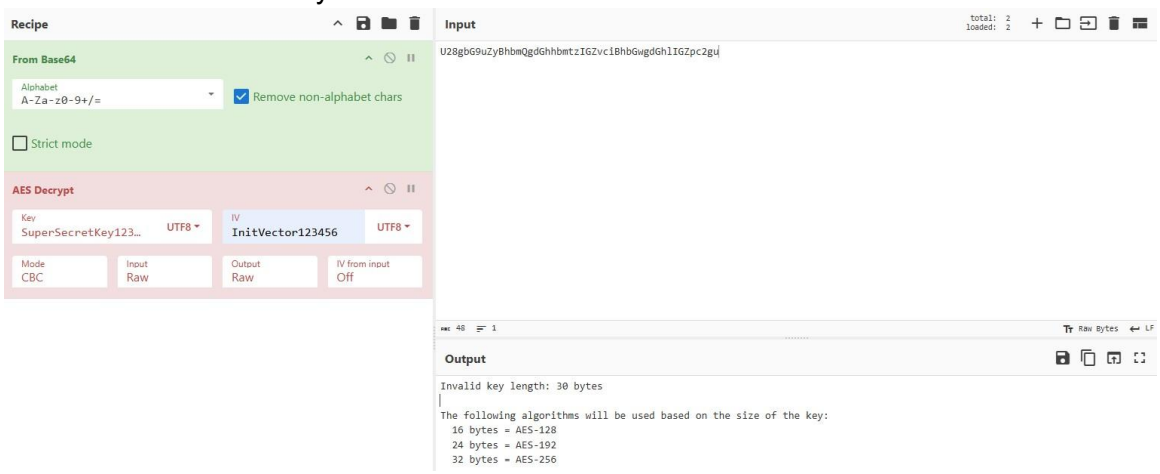


Figure 8. CyberChef reporting an invalid AES key length.

The recovered key was only 30 bytes long. Since AES-256 requires a 32-byte key, I tested a 32-byte candidate by appending two additional characters to the recovered key. This candidate successfully decrypted the payload and produced valid JSON.


```
{
  "instruction": "update_config",
  "encryption_type": "xor",
  "xor_key": 66,
  "payload_bytes": "2730223b1f2c50272f285532503b2a57303b5654565219"
}
```


This indicated that:

- the client was instructed to update its configuration,
- an additional XOR transformation was required,
- the encrypted data was stored in payload_bytes.

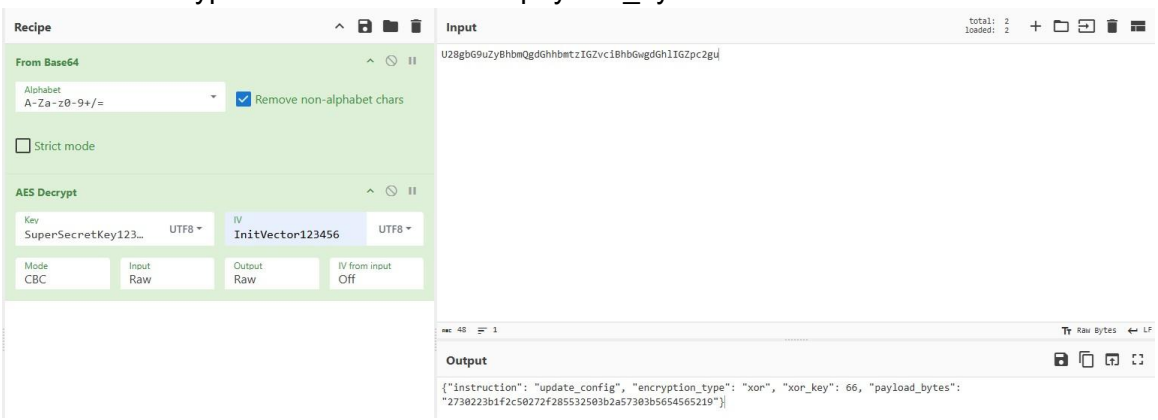


Figure 9. Successfully decrypted JSON recovered using CyberChef.

6.3 XOR Analysis

The decrypted JSON specified:

xor_key = 66

I decoded payload_bytes in CyberChef using XOR with key **66**.

However, the output did not produce meaningful plaintext.

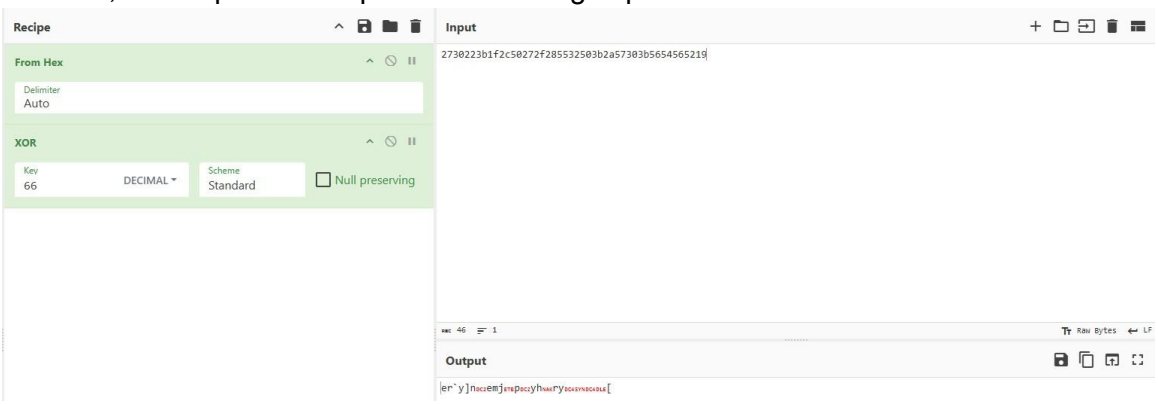


Figure 10. XOR decoding using key 66 produced unreadable output.

Because the recovered value did not produce a valid result, I decided to perform a systematic analysis instead of assuming the key was correct.

6.4 XOR Brute Force

I developed a Python script to test all **256 possible XOR keys**.
The script automatically checked each decoded result for:

- printable ASCII text,
- JSON,
- Base64,
- gzip/zlib,
- common CTF flag formats.

During the brute-force process, **XOR key 100** produced a readable result containing the challenge flag:

CTF_{H4CKL1V4_N3T_2026}

Figure 11. Python script identifying the correct XOR key and recovering the flag.

7. Conclusion

The investigation demonstrated a structured network forensic workflow. Sensitive configuration was recovered from anonymous FTP access, encrypted HTTP data was analyzed, multiple AES derivation methods were tested, and an automated XOR brute-force approach was used to recover the final CTF flag.